RAJALAKSHMI ENGINEERING COLLEGE
Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)
Department of Artificial Intelligence
V Semester B. Tech AIDS

AD23532 – Principles of Data Science

Academic Year: 2026-2027 **Batch:** 2024-2028

Ex 3a: Model Planning and Building – Classification

Name: Nischal **Date:** 06-10-2026

Register Number: 241801188 **Tools:** Python, Pandas, NumPy, Matplotlib, Seaborn, scikit-learn

Dataset: CustomerChurn_Preprocessed.csv (Telco Customer Churn, from Ex 2)

Aim

To plan, build and evaluate classification models that predict whether a telecom customer will churn: train Logistic Regression, Decision Tree and Random Forest classifiers on the preprocessed Telco data, compare them using the confusion matrix, Accuracy, Precision, Recall and F1-Score, identify the best-performing model, and use it to recommend actions that reduce customer churn.

Task 1: Load CustomerChurn_Preprocessed.csv (created in Ex 2)

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style='whitegrid')
pd.set_option('display.max_columns', None)

df = pd.read_csv('CustomerChurn_Preprocessed.csv')
print("Shape of the dataset :", df.shape)
print("Missing values       :", int(df.isnull().sum().sum()))
print("Non-numeric columns  :", df.select_dtypes(exclude='number').shape[1])
print("\nChurn distribution (0 = stayed, 1 = churned):")
print(df['Churn'].value_counts())
print((df['Churn'].value_counts(normalize=True) * 100).round(2))
```

Out [1]:

```
Shape of the dataset : (7043, 25)
Missing values       : 0
Non-numeric columns  : 0

Churn distribution (0 = stayed, 1 = churned):
Churn
0    5174
1    1869
Name: count, dtype: int64
Churn
0    73.46
1    26.54
Name: proportion, dtype: float64
```

> **Observation:** The file holds 7043 customers with 24 input features and the Churn target. It has no missing values and every column is numeric, so it can be used directly for modelling. Only 26.54% of the customers churned, so the classes are imbalanced and accuracy alone will not be a fair measure.

In [2]:

```
df.head(5)
```

Out [2]:

Columns 1–9 of 25

	gender	SeniorCitizen	Partner	Dependents	tenure	Contract	MonthlyCharges	TotalCharges	Churn
0	0	0	1	0	-1.277445	0	-1.160323	-0.994242	0
1	1	0	0	0	0.066327	1	-0.259629	-0.173244	0
2	1	0	0	0	-1.236724	0	-0.362660	-0.959674	1
3	1	0	0	0	0.514251	1	-0.746535	-0.194766	0
4	0	0	0	0	-1.236724	0	0.197365	-0.940470	1

Columns 10–14 of 25

	PaymentMethod_Bank transfer (automatic)	PaymentMethod_Credit card (automatic)	PaymentMethod_Electronic check	PaymentMethod_Mailed check	InternetService_DSL
0	0	0	1	0	1
1	0	0	0	1	1
2	0	0	0	1	1
3	1	0	0	0	1
4	0	0	1	0	0

Columns 15–20 of 25

	InternetService_Fiber optic	InternetService_No	PhoneService_Yes	MultipleLines_Yes	OnlineSecurity_Yes	OnlineBackup_Yes
0	0	0	0	0	0	1
1	0	0	1	0	1	0
2	0	0	1	0	1	1
3	0	0	0	0	1	0
4	1	0	1	0	0	0

Columns 21–25 of 25

	DeviceProtection_Yes	TechSupport_Yes	StreamingTV_Yes	StreamingMovies_Yes	PaperlessBilling_Yes
0	0	0	0	0	1
1	1	0	0	0	0
2	0	0	0	0	1
3	1	1	0	0	0
4	0	0	0	0	1

5 rows × 25 columns

Task 2: Split the dataset into Training (80%) and Testing (20%)

In [3]:

```
X = df.drop(columns='Churn')
y = df['Churn']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

print("Training set :", X_train.shape, "->", round(len(X_train) / len(X) * 100, 1), "% of the data")
print("Testing set  :", X_test.shape, "->", round(len(X_test) / len(X) * 100, 1), "% of the data")
print("\nChurn rate in training set : %.2f%%" % (y_train.mean() * 100))
print("Churn rate in testing set  : %.2f%%" % (y_test.mean() * 100))
```

Out [3]:

```
Training set : (5634, 24) -> 80.0 % of the data
Testing set  : (1409, 24) -> 20.0 % of the data

Churn rate in training set : 26.54%
Churn rate in testing set  : 26.54%
```

> **Note:** The split is stratified on Churn with `random_state=42`, so the training and testing sets keep the same churn rate and every model sees exactly the same data.

Task 3: Logistic Regression

In [4]:

```
lr_model = LogisticRegression(max_iter=1000, random_state=42)
lr_model.fit(X_train, y_train)
y_pred_lr = lr_model.predict(X_test)

print("Logistic Regression trained.")
print("Training accuracy: %.4f | Testing accuracy: %.4f" % (lr_model.score(X_train, y_train), lr_model.score(X_test, y_test)))
print("\nFirst 10 test customers (0 = stay, 1 = churn):")
display(pd.DataFrame({'Actual': y_test.values[:10], 'Predicted': y_pred_lr[:10]}).T)
```

Out [4]:

```
Logistic Regression trained.
Training accuracy: 0.8053 | Testing accuracy: 0.8034

First 10 test customers (0 = stay, 1 = churn):
```

	0	1	2	3	4	5	6	7	8	9
Actual	0	0	0	0	0	0	0	0	0	1
Predicted	0	1	0	0	0	1	0	0	0	0

2 rows × 10 columns

Task 4: Decision Tree Classifier

In [5]:

```
# An unrestricted tree is fitted first, only to show why its depth must be limited
dt_full = DecisionTreeClassifier(random_state=42).fit(X_train, y_train)
print("Unrestricted tree -> training accuracy: %.4f | testing accuracy: %.4f"
      % (dt_full.score(X_train, y_train), dt_full.score(X_test, y_test)))

# Final tree: depth limited to 5 to reduce over-fitting
dt_model = DecisionTreeClassifier(max_depth=5, random_state=42)
dt_model.fit(X_train, y_train)
y_pred_dt = dt_model.predict(X_test)

print("Depth-5 tree      -> training accuracy: %.4f | testing accuracy: %.4f"
      % (dt_model.score(X_train, y_train), dt_model.score(X_test, y_test)))
print("\nFirst 10 test customers (0 = stay, 1 = churn):")
display(pd.DataFrame({'Actual': y_test.values[:10], 'Predicted': y_pred_dt[:10]}).T)
```

Out [5]:

```
Unrestricted tree -> training accuracy: 0.9980 | testing accuracy: 0.7331
```

```
Depth-5 tree      -> training accuracy: 0.8039 | testing accuracy: 0.7991

First 10 test customers (0 = stay, 1 = churn):
```

	0	1	2	3	4	5	6	7	8	9
Actual	0	0	0	0	0	0	0	0	0	1
Predicted	0	1	0	1	0	0	0	0	0	0

2 rows × 10 columns

> **Observation:** The unrestricted tree memorises the training data (training accuracy 0.9980 against 0.7331 on the test set). Limiting the depth to 5 closes that gap (0.8039 against 0.7991), so the depth-5 tree is used as the Decision Tree model.

Task 5: Random Forest Classifier

In [6]:

```
rf_model = RandomForestClassifier(n_estimators=200, random_state=42)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)

print("Random Forest trained (200 trees).")
print("Training accuracy: %.4f | Testing accuracy: %.4f" % (rf_model.score(X_train, y_train), rf_model.score(X_test, y_test)))
print("\nFirst 10 test customers (0 = stay, 1 = churn):")
display(pd.DataFrame({'Actual': y_test.values[:10], 'Predicted': y_pred_rf[:10]}).T)
```

Out [6]:

```
Random Forest trained (200 trees).
Training accuracy: 0.9980 | Testing accuracy: 0.7835

First 10 test customers (0 = stay, 1 = churn):
```

	0	1	2	3	4	5	6	7	8	9
Actual	0	0	0	0	0	0	0	0	0	1
Predicted	0	1	0	0	0	0	0	0	0	0

2 rows × 10 columns

> **Observation:** The forest fits the training set almost perfectly (0.9980) but scores 0.7835 on unseen customers, so it over-fits with its default settings.

Task 6: Model evaluation

In [7]:

```
y_preds = {'Logistic Regression': y_pred_lr,
           'Decision Tree': y_pred_dt,
           'Random Forest': y_pred_rf}

rows = []
for name, pred in y_preds.items():
    cm = confusion_matrix(y_test, pred)
    tn, fp, fn, tp = cm.ravel()
    print(name)
    print(cm)
    print(f"TN = {tn}, FP = {fp}, FN = {fn}, TP = {tp}\n")
    rows.append({'Model': name,
                 'Accuracy': accuracy_score(y_test, pred),
                 'Precision': precision_score(y_test, pred),
                 'Recall': recall_score(y_test, pred),
                 'F1-Score': f1_score(y_test, pred)})

results = pd.DataFrame(rows).set_index('Model').round(4)
display(results)
```

Out [7]:

```
Logistic Regression
[[925 110]
 [167 207]]
TN = 925, FP = 110, FN = 167, TP = 207

Decision Tree
[[908 127]
 [156 218]]
TN = 908, FP = 127, FN = 156, TP = 218

Random Forest
[[919 116]
 [189 185]]
TN = 919, FP = 116, FN = 189, TP = 185
```

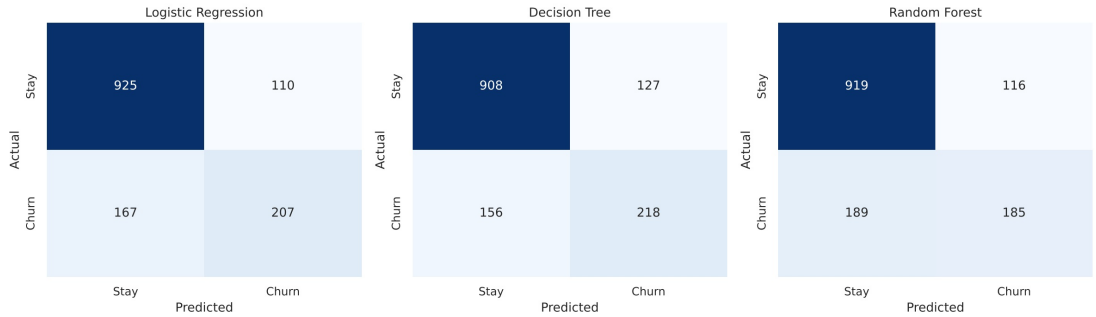
Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.8034	0.6530	0.5535	0.5991
Decision Tree	0.7991	0.6319	0.5829	0.6064
Random Forest	0.7835	0.6146	0.4947	0.5481

3 rows × 4 columns

In [8]:

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
for ax, (name, pred) in zip(axes, y_preds.items()):
    cm = confusion_matrix(y_test, pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False, ax=ax,
                xticklabels=['Stay', 'Churn'], yticklabels=['Stay', 'Churn'])
    ax.set_title(name)
    ax.set_xlabel('Predicted')
    ax.set_ylabel('Actual')
plt.tight_layout()
plt.show()
```

Out [8]:



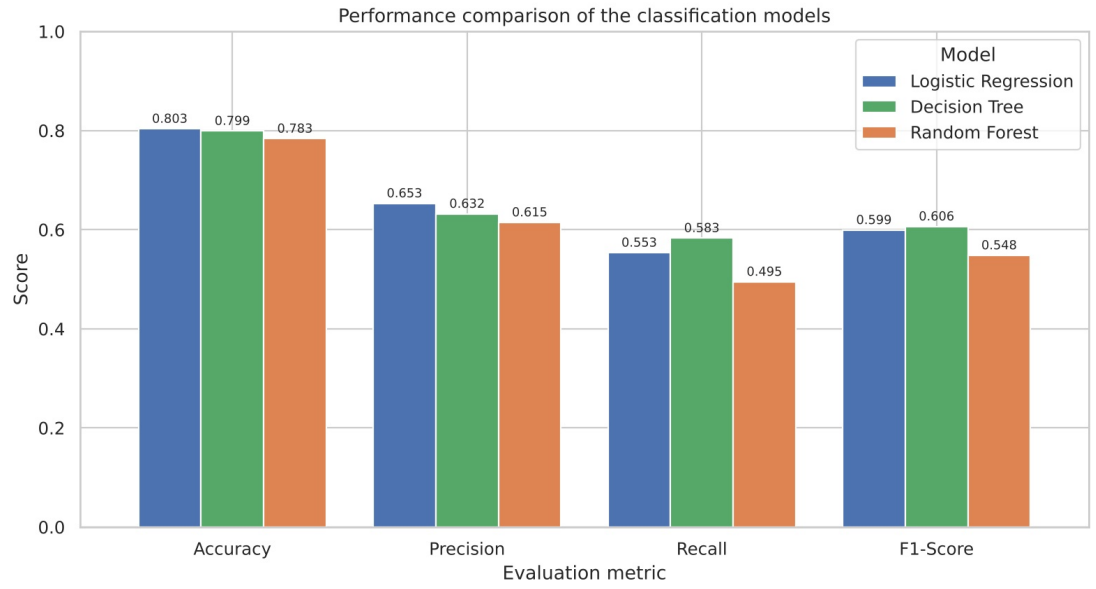
Observation: In the confusion matrices, the models miss between 156 and 189 of the 374 customers who actually churned (false negatives), and wrongly flag between 110 and 127 loyal customers (false positives). Accuracy looks good (78.3%–80.3%) mainly because most customers stay, so Precision, Recall and F1-Score give a fairer picture of churn detection.

Task 7: Compare the three models (bar chart)

In [9]:

```
ax = results.T.plot(kind='bar', figsize=(10, 5.5), width=0.8,
                    color=['#4C72B0', '#55A868', '#DD8452'])
for container in ax.containers:
    ax.bar_label(container, fmt='%.3f', fontsize=8, padding=2)
ax.set_title('Performance comparison of the classification models')
ax.set_xlabel('Evaluation metric')
ax.set_ylabel('Score')
ax.set_ylim(0, 1)
plt.xticks(rotation=0)
plt.legend(title='Model', loc='upper right')
plt.tight_layout()
plt.show()
```

Out [9]:



Task 8: Best-performing model

In [10]:

```
ranking = results.sort_values('F1-Score', ascending=False)
best_name = ranking.index[0]
display(ranking)
print("Best model (highest F1-Score):", best_name)
print("Highest Accuracy  :", results['Accuracy'].idxmax())
print("Highest Precision :", results['Precision'].idxmax())
print("Highest Recall    :", results['Recall'].idxmax())

models = {'Logistic Regression': lr_model, 'Decision Tree': dt_model, 'Random Forest': rf_model}
```

Out [10]:

Model	Accuracy	Precision	Recall	F1-Score
Decision Tree	0.7991	0.6319	0.5829	0.6064
Logistic Regression	0.8034	0.6530	0.5535	0.5991
Random Forest	0.7835	0.6146	0.4947	0.5481

3 rows × 4 columns

```
Best model (highest F1-Score): Decision Tree
Highest Accuracy  : Logistic Regression
Highest Precision : Logistic Regression
```

```
Highest Recall    : Decision Tree
```

> **Result:** The **Decision Tree** is the best-performing model, with the highest F1-Score (0.6064) and the highest Recall (0.5829). F1-Score is used to decide because only 26.5% of customers churn, so accuracy alone rewards a model that ignores most churners. The runner-up, Logistic Regression, is very close (F1-Score 0.5991, a gap of only 0.0073) and has the highest Accuracy (0.8034) and Precision (0.6530), while the Random Forest is weakest (F1-Score 0.5481).

Task 9: Business recommendations

The recommendations are based on the best-performing model. The chart shows the factors that the model relies on most to separate churners from loyal customers, and the table that follows checks each factor against the original data.

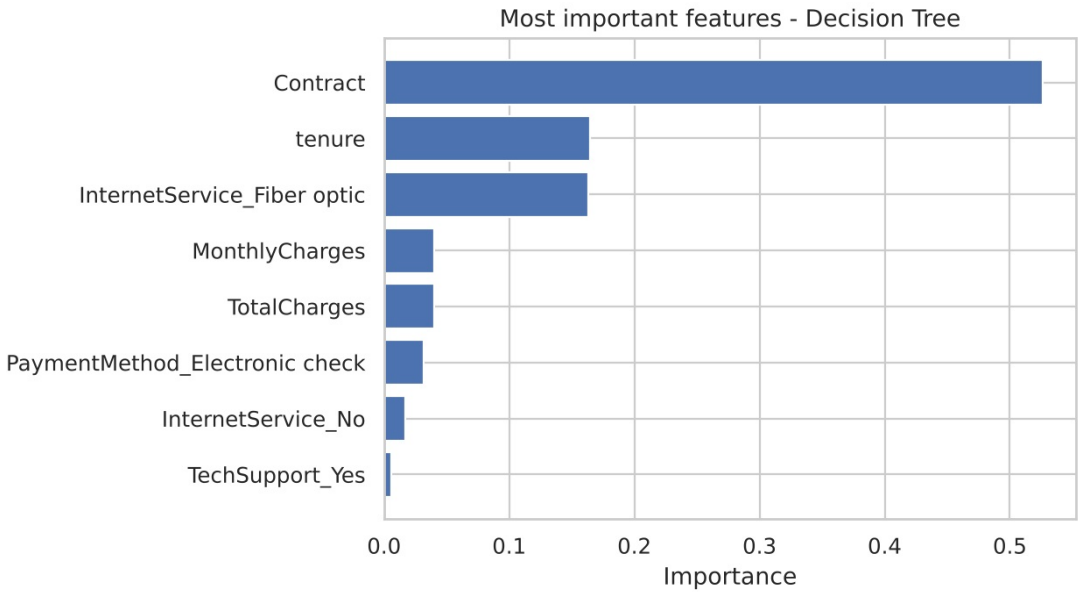
In [11]:

```
best_model = models[best_name]
if hasattr(best_model, 'feature_importances_'):
    importance = pd.Series(best_model.feature_importances_, index=X.columns)
else:
    importance = pd.Series(np.abs(best_model.coef_[0]), index=X.columns)
top_imp = importance.sort_values(ascending=False).head(8)
print(f"Top 8 features used by the {best_name}:")
print(top_imp.round(3))

plt.figure(figsize=(8, 4.5))
plt.barh(top_imp.index[::-1], top_imp.values[::-1], color='#4C72B0')
plt.title(f'Most important features - {best_name}')
plt.xlabel('Importance')
plt.tight_layout()
plt.show()
```

Out [11]:

```
Top 8 features used by the Decision Tree:
Contract                          0.527
tenure                            0.164
InternetService_Fiber optic       0.163
MonthlyCharges                    0.040
TotalCharges                      0.040
PaymentMethod_Electronic check    0.032
InternetService_No                0.017
TechSupport_Yes                   0.005
dtype: float64
```



In [12]:

```
# Check the top factors against the original (unprocessed) customer data
raw = pd.read_csv('Telco-Customer-Churn.csv')
raw['TotalCharges'] = pd.to_numeric(raw['TotalCharges'], errors='coerce')
churned = (raw['Churn'] == 'Yes')

evidence = {
    'Month-to-month contract':       churned[raw['Contract'] == 'Month-to-month'].mean() * 100,
    'Two-year contract':             churned[raw['Contract'] == 'Two year'].mean() * 100,
    'Tenure of 12 months or less':   churned[raw['tenure'] <= 12].mean() * 100,
    'Tenure of more than 12 months': churned[raw['tenure'] > 12].mean() * 100,
    'Fiber optic internet':          churned[raw['InternetService'] == 'Fiber optic'].mean() * 100,
    'DSL internet':                  churned[raw['InternetService'] == 'DSL'].mean() * 100,
    'Tech support (internet users)':    churned[raw['TechSupport'] == 'Yes'].mean() * 100,
    'No tech support (internet users)': churned[raw['TechSupport'] == 'No'].mean() * 100,
}
print("Churn rate (%) by customer group:")
for group, rate in evidence.items():
    print(f"  {group:36s}: {rate:5.2f}")
```

Out [12]:

```
Churn rate (%) by customer group:
  Month-to-month contract             : 42.71
  Two-year contract                   :  2.83
  Tenure of 12 months or less         : 47.44
  Tenure of more than 12 months       : 17.13
  Fiber optic internet                : 41.89
  DSL internet                        : 18.96
  Tech support (internet users)       : 15.17
  No tech support (internet users)    : 41.64
```

Three recommendations for the company

1. **Move customers from month-to-month to longer contracts.** Contract type is the model's strongest signal (importance 0.53). Month-to-month customers churn at 42.71% against 2.83% on two-year contracts, so offer discounts, loyalty rewards or free upgrades for a one- or two-year commitment, starting with the month-to-month customers the model flags as likely to leave.
2. **Run a retention programme for the first year.** Tenure is the next strongest signal (importance 0.16), and customers in their first 12 months churn at 47.44% against 17.13% afterwards. Add a welcome and onboarding call, an early-life offer and regular check-ins, and score new customers with the model each month.
3. **Improve the fibre-optic experience and bundle support services.** Fiber optic internet is also a top factor (importance 0.16), with 41.89% churn against 18.96% for DSL. Investigate fibre price and service quality, and include tech support or online security in fibre plans: customers with tech support churn at 15.17% against 41.64% without it.

The best model finds about 58% of the customers who churn, with about 63% of the customers it flags actually leaving, so retention offers should be low-cost and well targeted.

Result

Thus Logistic Regression, Decision Tree and Random Forest classifiers were trained on 80% of the preprocessed Telco Customer Churn data (7,043 customers) and tested on the remaining 20%. They were compared using the confusion matrix, Accuracy, Precision, Recall and F1-Score. The Decision Tree was the best-performing model (F1-Score 0.6064, Recall 0.5829), with Logistic Regression a close second. Based on the factors this model relies on most, three recommendations were made: move customers to longer contracts, run a first-year retention programme, and improve the fibre-optic experience while bundling support services.